

Lec 16 Numerical Example, Limitations of AE

Lec 16 Numerical Example, Limitations of AE

Lecture	16 / 63 + intro
Duration	27:25
Captions	English (auto-generated)
Words	4,126
Video	vvkdhI5ECHM
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the lecture series on fundamentals of generative AI and the large language models. We are in the week two and today is the last theoretical session of week two. And in this particular session, we will see the step-by-step the working mechanism of autoencoder. We will see this with one numerical example and then we will end this particular week two with the limitations of autoencoders. So, we will first start with a numerical example.

[00:32] This is what is the architecture we have. We here five indicates we have totally five features and the there are two hidden layers and the first hidden layer is having a four neurons. The next hidden layer is having three neurons. So, then the this is an encoder. And from this three, what we are getting is the latent representation. Now, this latent representation we are passing to and this entire is the decoder. So, we will understand the overall once again the autoencoder with one numerical example starting off from input to end a latent representation. So, now since we have a five features, let us consider this particular example. These are our five features.

[01:31] Now, if you look into because it is five features and it is a passed to a hidden layer which is having four neurons. So, let me show you diagrammatically. These

are five input features and this is passed to a hidden layer which is having a totally four neurons. So, hidden layer is having neurons. So, how is that the calculations are going to be? First, we have to consider the weights. And then, since it is a fully connected neural network, so you will be connecting this input to with this. And since it is this is 0.8, 0.4, 0.6, 0.2, and 0.5.

[02:19] This is passed to this neuron. We are establishing a connection like this. Right? So, what exactly we are doing? Input is 0.8, which gets multiplied with this weight. And the weight here is 0.2. And 0.4 gets multiplied with a weight, and the weight is 0.5. Similar way, 0.4, 0.3, 0.1. And then we know that there will be a bias at term that will be added, and the bias here is 0.01. So, we can able to understand this diagrammatically, right? So, what we are doing? Now, the input is passed to a first hidden layer, which is having totally four neurons. So, how the calculation is going to be? Is it not the calculation is this? We have an input 0.8 getting multiplied with its weight 0.2.

[03:12] Plus 0.4 getting multiplied with its weight 0.5. Then 0.6 getting multiplied with 0.4, 0.2 with the weight 0.3, 0.5 with the weight 0.1. Plus the bias at term 0.01. So, perform the calculation, and then we are going to get the summation term as 0.72. Then, there is an activation function. And since it is a hidden layer, we have seen so far that for a hidden layer we'll be using a non-linear activation function. So, here the activation function that we are using in this particular example is ReLU activation.

[03:54] What is the formula for a ReLU? ReLU is max of 0, x. So, now 6 here x here is 0.72. So, max of 0, 0.72 it is 0.72. So, the output of this particular neuron is 0.72. Next, what is that we are going to do for this neuron? Is it not Let me change the color so that you will have a clarity. » [snorts] » Okay. Now, this is connected to this. This input is connected to this neuron. This input is connected to this. This input is connected to this. And this input is connected to this. So, again we require the five weights. And those five weights are these.

[04:37] One is 0.1, 0.3, 0.2, 0.6, and 0.4. So, is it not we get this particular calculation? We are going to have this is a second neuron. And then we are multiplying the input with its weight. So, 0.8 getting multiplied with 0.1, 0.4 with 0.3, and so on so forth. And this is what is the summation term. Then you're passing it to an activation. So, max of 0, x 0.65. And the maximum between these two is 0.65. So, did you understand how exactly the calculations is being made? Similar way, next we are going to establish the connection to the next neuron. So, this input to this, this input here this and here.

[05:28] So, once again you will do a same calculation. So, here the And we are now considering these set of weights. And the similar way and the last neuron we are passing the input and the connections will be made in this week. Since there are five weights and those are five weights are the last weights. So, and uh this is what is

the calculation that we are getting. So, max of 0.77 is 0.77. So, now if you observe here, what exactly we have done? Uh the input is a passed to a first hidden layer. And how many neurons are there in the first hidden layer? We have four neurons. So, the input that we have passed is this.

[06:19] And finally, we have compressed this five input features to the four features. Is it not we got the four values here? So, these four are the output of the first hidden layer. Is it not we have compressed the information? Right? Okay. Now, let us take the next cycle where from five you have passed it to four, from four it is now three. Right? So, which means how is that the architectural diagram looks like? So, let me take and let me draw the architecture diagram here. We have four hidden neurons.

[07:01] Right? And that is now passed to a three. So, we have now three neurons. So, which is what from a four we are reducing it to three. This is the part that we are discussing. Now, how is that the first connection you will establish? From here, the connection is established to this. From this, you are establishing a connection. Is it not this way we will do a calculation? And of course, there is a bias term. So, we require how many weights we require? We require four weights. Where are those four weights? These are those four weights. So, now what is this value? Whatever we have obtained in the previous. So, this is what is the previous value, which is written here.

[07:46] So, one is 0.72, this is 0.65, this is 0.66, and this is 0.77. Yes? Okay. Now, what are these weights? The weights are 0.3, 0.5, 0.2, and 0.1. So, is it not this is the calculation we are going to do? » [snorts] » The input is getting multiplied with and then we are adding the bias at term. This is the bias at term that we have. So, this is also an hidden layer. Because it is an hidden layer, we are once again using an ReLU activation function. And what is the ReLU? It is max of 0, X. And max of 0 with 0.760, the maximum between these two is this value. Right? Now, similar way, what is that second we will be doing? Let me change. So, here this is a connected to this. This input is connected. Similar

[08:46] way, what we have seen in the previous slide, the same calculation. Now, there are four weights that are needed, and those four set of weights are these. And this is the calculation that we are doing it. Right? So, if there are any calculation mistakes, if there are any typos, I request the learners to correct that. And the next is we are going to establish a connection to this particular uh neuron. So, these are four weights. And this is what is the calculation. So, for the third, you are multiplying input 0.72 with the weight, and you are totally you are adding the bias at term. So, with this, what exactly we have done? Is it not Uh initially started with the five, then we have reduced it to four, now we have

[09:39] reduced it to three. So, what is the final outcome that we have obtained? The final outcome that we have obtained is this. What What What is What are these

values? The first value is this, second value is this, and the third value is this. What did we do? From five, we have passed it to four, from four we have passed it to three. And this is your input. This input is now compressed to these particular values. So, nowhere we have deleted any values. We have systematically compressed the values.

[10:15] Now, since this is your hidden last hidden layer, the output of this hidden layer is the latent representation. So, can we say that this is the latent representation now? Answer is yes, right? Now, what we are doing? From three, this is three, from three we are now passing it to four. So far we have seen the encoder. So, now we are passing it to a decoder. And the decoder here is having totally four neurons. Now, how is that we will be establishing the connection? Is it not this is connected to this, and this feature is connected to this neuron, and this feature is connected to this particular neuron. So, how many weights are required? We require three weights.

[11:06] And which are those three weights? These are the three weights. So, how is that you will be establishing the connection? Now, since you have already understood how we have done, I'm showing all of the connections at a calculations at a time. So, since here this is 0.760, 0.714, and this is 0.944, you are multiplying with its associated weight. This is 0.4, this is 0.2, and this is 0.3. Right? So, we are establishing the connection. We are multiplying input with its weight, and we are adding the bias at term. Uh and here also, if you see, we are passing it to an hidden layer. When you are passing it to an hidden layer, uh there has to be an activation function. You have a summation, and you have an activation,

[11:57] and that activation is a ReLU. So, max of 0, x, and x here is 0.40. Maximum between these two is 0.74. Similar way, you will establish the connection from this input to this neuron, from this input to this neuron, and from this input to this neuron. We have a totally three weights, and those are three weights are this. Similar way, you will continue with whatever we have learned, and these are the four calculations that you will be obtaining.

[12:30] And what exactly we have done? Here, uh the input was a three. From three, you have passed to an hidden layer, which is having a totally four neurons. So, therefore, from the input to three, we are now getting four. Why did we do this? Because what is our ultimate aim? The aim is to reconstruct. Uh when we are reconstructing, we should go back to again the five features. So, systematically, we have increased from three to four. Now, the last step here is from a four, you are now passing it to two, five. Now, uh now this is the last output layer, right? What is the original input that we have considered? The original input what we have considered here is a continuous values. If that is the case,

[13:19] what should be our output? The output also should be a continuous value. If the output is also a continuous value, then what should be the activation function that

we have to use at the hidden layer? Go back and then recall the activation function has to be linear activation function. Therefore, at an output layer, the activation function that we are using here is linear. Why linear? Because the input what we have considered isn't continuous value. Obviously, the output also should be a continuous value only.

[13:50] So, now from four you have to increase to five. How will be the architectural diagram? These are the four neurons and from here you have to get the five values. So, totally you will have a five. How is that you will be establishing the connection? This input is connected to this. This input is connected to this. This input is connected and this input is connected to this. So, how many weights are required? We require four weights. Those four weights are these.

[14:18] And then there is a bias at them. Similar way, second you will establish a connection in this way. Uh this input to this, from this input. This is what is the second connection. And those weights are these. Similar way, we will establish a connection to the third neuron. And those are these set of weights. And so on so forth. So, this is what is the overall calculation. So, for the first set of calculation, now you can see after we have performed this particular calculation, this is the value that we have obtained. That has to be passed to an activation function. What is an activation function here? f of x equal to x or f of a equal to a . Now, what is that a value? It is 0.67838.

[15:12] So, when you pass, you are going to get the same value. Right? So, that is the reason activation function part is not being shown. That is uh here. Because whatever the value that you pass it to an activation function, and because it is a linear activation now function, the output will be the same. Same as whatever you have computed here. So, this is uh the second output. And uh this will be the third output, fourth output, and then the fifth output. Now, what is that we have done? So, uh initially, the input what we have considered was a five. From five, you have reduced it to four. From four, we have reduced it to three. So, the latent uh representation here is a three. From a three, we have again systematically

[15:57] increased. From three, we reached to four. From four, we have reached to five. This is what is the output that we have obtained. Now, this output, is it not an reconstructed values, right? So, look at this. The original input what we had is a 0.8, 0.4, 0.6, 0.2, and 0.5. And these are reconstructed uh values. Now, what is the next step? The next step has to be the uh error calculation. Since here, it is a continuous values, what should be the error uh function? Definitely, yes, recall. It's a mean squared error.

[16:35] So, we are considering mean squared error. And since there are five values, you are taking uh the average of all of this. You will add up, and then you will uh take the average. So, this is what is the error value that we have obtained. What are we

getting from this? What did we understand? So, once the error we computed, once the error is more, what we do? We backpropagate during the training uh process, you will backpropagate it. When you backpropagate, uh we are using an optimizers. What is the role of an optimizer? The role of an optimizer has to update the weights. So, which weights it will update? Definitely, all the weights which are uh from five to four, four to three, three to four, four to

[17:19] five. All those weights will be optimized over a period of time. So, by end of training, what is that we are doing? We are updating the weights and we are also updating the bias here. So, by doing this towards the end of the training, are you now convinced that the weights what we are going to obtain will be the optimized weights? Do you agree to this? Yes? Now, once we get the optimized weights, what is the next step? Now, with that optimized weight, when we pass an input, input when it is getting multiplied with the optimized weight, of course there is a bias term. So, we are going to get the optimized hidden representation. This optimized hidden representation, when you pass it to two and decoder, so the decoder is again

[18:12] using the optimized weights. Now, what reconstructed value that we are going to get will be definitely matching to the original input. This is what is the overall working mechanism of autoencoders. I hope this is clear. So, quick summary, we have an encoder, we have an code or an latent representation, and we have an decoder. So, here we have used an ReLU as an activation function. At the output layer, we will be here again ReLU will be using. At an output layer, based on what kind of values we are generating or reconstructing, accordingly, not generating, I'm correcting the word, it is reconstructing.

[19:01] We'll be using the activation function. Suppose if the input is you know, continuous values, like this a continuous value, then at the output layer we'll be using a linear activation function and the error we will be using is a mean squared error. Now, if instead of this, if you have a values in terms of zero and one, you're doing so you're dealing something with the binary, then they at the output we'll be using a sigmoid and then we will be using an binary cross entropy loss function.

[19:34] So, towards the end of the training, our aim is to update the weights and then the bias. We get the optimized weights and optimized bias. With that optimized weights and the bias, when we perform the calculation, we get very good latent representation. With that very good latent representation, we are going to get a very good reconstructed values. Then when we calculate the error, the error will be as minimum as possible, which means the reconstructed values are closer to the original input.

[20:08] With this, we have come to an end of this particular session. Before we wind up, let us see the last topic, that is the limitations of autoencoders. So, here we are reconstructing the data. So, what exactly is the limitation? If you have understood the

overall working mechanism of autoencoders, the entire calculation or a reconstruction is happening because from this particular latent representation. The encoder has to give us the good representation and [snorts] from that latent representation only the reconstruction is happening. So, do you all agree that whatever is happening is all happening from this latent representation. Now, look at the limitations there.

[20:57] Now, for example, you have obtained a very good reconstructed value and after you obtained a reconstruction one, if you compare it to the original input, you are seeing that the error is very low. Now, with this, what are we concluding? We are concluding that we are getting very good reconstructed one. But, what about the latent representation? From that the latent representation, can we infer some information? Can we get some information? The answer is no. Why?

[21:26] Even though the autoencoder has reconstructed the input with very low reconstruction error, and the error is calculated using this, even with very low reconstruction error, if you get the output, there is not there is no guarantee that the model has learned the meaningful features. There is no guarantee with that. What do you mean by the model has learned the meaningful features? If you take the handwritten example, handwritten digit images, from this latent representation, can you able to tell whether the number is two or nine? Whether the stroke is a thin or a thick? Whether the image is a slant image or not? From the latent representation, there is nothing that we are going to get that information. So,

[22:13] this is the first limitation. Although the reconstructed image is very good with the low reconstruction error, latent representation, we cannot infer anything from that. Second, just because the reconstructed error is very low, if you take latent representation, and if you if you want to perform some operations like classification or prediction or clustering, anomaly detection, any other operation that you want to do just with that latent representation, it is not possible. Why?

[22:48] » [laughter] » You may ask, reconstruction error is very less. So, which means this is very powerful. We cannot tell that. Just because the reconstruction error is less, if you rely on that latent representation, it has not captured the information which is human readable form. So, with that you cannot perform any other further operations such as classification, prediction, clustering, anomaly detection. No. And next this reconstruction you have obtained from this latent representation, not directly from the original input. So, whatever you have reconstructed, it is always an approximation, but not the exact. That is the third limitation.

[23:32] Since the reconstructed output is generated from the compressed latent representation, the decoder always produces an approximation of the input, » [snorts] » the model will not generate a new sample here. So, that is the third limitation. And the fourth limitation here is once again everything is dealing with the latent space only, right? Now, what will happen if you increase the latent space? And what will

happen if you decrease the latent space? I think we have discussed this somewhere in one of our previous sessions also, but since this is the final limitation that we are discussing, I'm re-bringing that point once again here. So, what is a limitation? Now, say for example, if you reduce the size of a latent space,

[24:14] can you able to see this diagram? When you when you keep the latent space size as a small, then too much information is getting compressed. So, when too much information is compressed, definitely the information loss is there. That's what the first case this is. Too much of information loss is there. Because of this, whatever reconstruction, can you see it is a poorer reconstruction one. Okay. Now, let's take the contradictory.

[24:42] What is the contradictory one? What if we increase the latent space? If you increase the latent space, definitely here we are getting a very good reconstruction one. But, there is one possibility. Maybe the model has just a copied the values. It has not learned any robust features possibility, right? So, the second case here is definitely reconstruction error is low. Why? Because the values are very much a matching to the original ground truth here. So, there is a low reconstruction error, no doubt. But, the possible problem here is maybe the model has not learned There is a weak learning feature learning. One more example if I have to give you, let us assume there are three input features and there are

[25:30] uh five hidden. So, and output you should get the three back. So, there is a very high possibility that it has just copied. So, because it has just copied, it is giving you the reconstructed values also, which is very close to the original. But, where is the learning here? And these values can be just zero. So, that is a problem in latent representation if you increase the size. So, this is the overall problem with autoencoders.

[25:58] This problem is mainly because of the latent representation there. So, the this leads to a topic called variational autoencoders, where it addresses all of these limitations. How? By introducing a regularization to a latent space. We are going to bring a regularization to a latent space and that regularization is by adding an KL divergence to a loss function. So, here you just have a reconstruction term in the loss, but when you take an VAE, you will also have an KL divergence term. And this is a mainly to take care of the latent space. So, this will be our point of discussion, that is variational autoencoder, in the coming week. And in the next session, you will will since you are now reaching to an

[26:51] end of week two, this is the last theoretical session and in the next session you will see an hands-on implementation on different types of VAE. When I say different types, what are types here? Can you recall from our one of our sessions? It can be an shallow autoencoder, it is an deep autoencoder, it is an convolutional autoencoder. All of this you will be seeing this in our next session. With this, thank you.